

Docente: JEFFERSON DE OLIVEIRA CHAVES

Discente: VITOR GABRIEL CAGOL ESCOBAR

Lista de exercícios número 2

Disciplina: Web III

Foz do Iguaçu, Paraná 2026

Exercício 2 (Resposta):

c) Ciclo de Vida do Controller (LifecycleController)

No Spring Boot, os Controllers são gerenciados por um container de Inversão de Controle (IoC). Por padrão, eles são criados usando o padrão *Singleton*, o que significa que existe apenas *uma única instância* desse Controller em toda a aplicação.

i) Quantas vezes apareceu a mensagem “Chamou o método init”?

Apareceu apenas 1 vez. O método anotado com `@PostConstruct` é executado logo após o Spring instanciar o `LifeCycleController` e injetar suas dependências (durante a inicialização da aplicação, antes de receber qualquer requisição).

ii) Quantas vezes apareceu a mensagem “Chamou o método service”?

Aparecerá 1 vez para cada vez que você acessar a rota `/lifecycle`. Se você acessou apenas uma vez para testar, apareceu uma vez.

iii) Quantas vezes apareceu a mensagem “Chamou o método destroy”?

Aparecerá 1 vez, mas apenas no momento em que você *encerrar a aplicação* (por exemplo, parando a execução no VSCode ou pressionando `Ctrl+C` no terminal). O `@PreDestroy` é chamado para limpeza de recursos antes do objeto ser destruído pelo container.

iv) Atualizando seu navegador muitas vezes, apertando F5 por exemplo, existe algum efeito?

Sim. O método `init` não será chamado novamente, mas a mensagem “Chamou o método service” aparecerá no console toda vez que você apertar F5. Isso ocorre porque cada F5 envia uma nova requisição HTTP GET para o servidor, acionando o método mapeado por `@GetMapping`.

d) Compartilhamento de Estado e Concorrência (ContadorController)

Aqui entramos em um dos conceitos mais críticos do desenvolvimento web: o estado da aplicação e a concorrência. Como vimos, o Spring cria apenas *uma*

instância do Controller (Singleton). Logo, o atributo `private int contador = 0;` pertence a essa única instância e é *compartilhado por todas as requisições*.

i) Atributos de classe podem ser compartilhados entre diferentes requisições dentro da aplicação. Em quais situações isso pode ser útil?

É útil quando precisamos manter informações globais da aplicação em memória. Exemplos incluem: manter um pool de conexões com o banco de dados aberto e reutilizável, armazenar configurações gerais do sistema, ou implementar sistemas de cache simples onde dados que não mudam com frequência são guardados na memória para acesso rápido de todos os usuários.

ii) Na sua opinião, essa abordagem é adequada para sistemas com múltiplos usuários acessando o sistema simultaneamente?

Não, não é adequada. Como a variável `contador` está sendo modificada (`contador++`) por múltiplas requisições ao mesmo tempo, ocorre um problema clássico chamado *Condição de Corrida (Race Condition)*. Se dois usuários acessarem a rota no exato mesmo milissegundo, ambos podem ler o valor `5`, incrementar para `6` e salvar `6`, perdendo uma das contagens (o certo seria chegar a `7`). Para fazer isso de forma segura no Java, seria necessário usar classes atômicas (como `AtomicInteger`) ou blocos sincronizados.

iii) As informações de uma requisição não são compartilhadas. Dizemos que são "Thread Safe". Explique a importância desse comportamento.

Dizemos que algo é "Thread Safe" (Seguro para Threads) quando funciona corretamente mesmo com vários usuários (threads) acessando ao mesmo tempo. Variáveis criadas *dentro* dos métodos do Controller (variáveis locais) são seguras porque cada requisição roda em sua própria Thread, e cada Thread tem sua própria área de memória (Stack). A importância disso é garantir a *integridade e segurança dos dados*. Se as requisições não fossem isoladas, o Usuário A poderia acidentalmente ver ou sobrescrever os dados bancários do Usuário B que fez uma requisição no mesmo momento. Isolamento garante previsibilidade no sistema web.